

CoachAI — File Structure Guide

Where every file lives, and what it's responsible for.

The 4 questions you'll get asked, answered up front:

- **Main file / entry point:** `run.py` (root folder) — running `python run.py` starts the whole app.
- **Main UI file:** `src/ui/app.py` — all Streamlit tabs/pages are wired here.
- **Core control logic:** lives in `src/core/` — orchestrator, config, LLM client, prompts.
- **Scores / reports:** generated into `data/reports/` (one JSON per session) after each session ends.

1. Root Folder

File	What it does
<code>run.py</code>	The main/entry file. Launches the Streamlit server.
<code>start.bat</code>	Windows double-click launcher — activates the venv and runs <code>run.py</code> .
<code>streamlit_app.py</code>	Alternate entry point used by Streamlit Cloud deployment.
<code>README.md</code>	Full project documentation and file map.
<code>requirements.txt</code>	List of all Python package dependencies.
<code>.env</code> / <code>.env.example</code>	Where the <code>GROQ_API_KEY</code> (LLM credential) is configured.

2. `src/core/` — Control Room

Owens orchestration, configuration, the LLM client, and every prompt.

File	What it does
<code>orchestrator.py</code>	Top-level controller: starts a session, routes each turn, ends session, builds report.
<code>config.py</code>	All app settings — file paths, escalation thresholds, model name — loaded from <code>.env</code> .
<code>llm.py</code>	Wraps the Groq API call, with the model fallback chain (70B -> 8B -> Gemma -> Llama3-8B).
<code>models.py</code>	Pydantic data schemas: <code>Message</code> , <code>TurnAnalysis</code> , <code>SessionState</code> , <code>PerformanceReport</code> , etc.
<code>prompts.py</code>	Every single agent's LLM system prompt, centralized in one file.
<code>database.py</code>	Saves/loads sessions and reports to the SQLite file <code>data/coach.db</code> .

3. `src/modules/` — Per-Turn Business Logic

Sits between the Orchestrator and the individual agents.

File	What it does
<code>conversation_manager.py</code>	Runs the fixed agent sequence every turn and assembles the <code>TurnAnalysis</code> result.
<code>session_config.py</code>	Handles interaction mode selection: Simulator / Manual / Replay.

performance_analytics.py	Aggregates scores/trends across multiple past sessions.
hall_of_fame.py	Archives the best- and worst-scoring sessions.
survival_game.py	Powers the multi-ticket 'arcade' practice mode.

4. src/agents/ — 24 Specialized AI Agents

Each file is one self-contained class with one responsibility.

File	What it does
customer_simulator.py	Generates realistic AI customer messages, turn by turn.
intent_sentiment.py	Classifies customer intent, sentiment, frustration level, trend.
knowledge_recommendation.py	RAG: retrieves matching KB chunks + LLM-synthesizes one tip.
coaching_suggestion.py	Suggests the agent's reply; scores tone and clarity.
escalation_monitor.py	Rule-based scoring of escalation/flight risk (no LLM call).
post_interaction_summary.py	Builds the end-of-session report: sentiment journey, resolution score.
coach_calibrator.py	Learns per-agent when a coaching tip should actually be shown.
predictive_csat.py	Forecasts live CSAT (1-5) and churn risk %.
manager_supervisor.py	Simulates a supervisor whisper-note or takeover on extreme risk.
compliance_monitor.py	Flags agent replies that contradict the knowledge base.
tone_rewriter.py	Rewrites a rough agent draft into a polished, empathetic reply.
auto_kb_agent.py	Auto-drafts a new FAQ entry when a knowledge gap is detected.
auto_pilot_agent.py	One-click autonomous reply generation plus a backend action.
competitor_defection_agent.py	Detects 'I'll switch to a competitor' churn threats.
customer_mind_reader.py	LLM guess at the customer's unstated internal thoughts.
multiverse_simulator.py	'What if the agent replied differently?' branching demo.
viral_threat_predictor.py	Flags risk of the customer posting publicly / going viral.
fraud_detector.py	Flags refund-abuse or scam patterns.
jira_bug_generator.py	Converts a transcript into a structured engineering bug ticket.
scenario_generator.py	LLM-generates new training scenarios on demand.
qa_audit_agent.py	Rule-based ISO-style pass/fail audit of a session report.
cognitive_load_agent.py	Rule-based estimate of the agent's mental workload.
patience_clock_agent.py	Rule-based estimate of turns remaining before customer drop-off.
bot_agent.py	Scripted first-line chatbot that runs before a human agent joins.

5. src/rag/ — Knowledge Base Search

File	What it does
------	--------------

knowledge_base.py	Chunks ingested files (~512 chars) and does keyword-overlap search.
ingest.py	Validates and dispatches uploaded files by type (txt/pdf/docx/json/etc.).

6. src/ui/ — The Streamlit Interface

File	What it does
app.py	THE main UI file — page routing and every tab in the app.
panels.py	Renders the conversation window and the live coaching panel.
zomato_widgets.py	Order card, delivery-rider tracking, and bot-chat widgets.
avatars.py	Avatar icon rendering helpers.

7. src/tools/ and src/api/

File	What it does
tools/mock_backend.py	Fake order-lookup / refund / voucher APIs used for the live demo.
api/server.py	Optional REST API entry point (outside the Streamlit UI).

8. data/ — Where All Scores and Content Live

File	What it does
data/reports/	One JSON file per completed session — this is where 'scores' are saved.
data/coach.db	SQLite database storing session state.
data/coach_calibration.json	Per-agent history of when coaching tips were shown/suppressed.
data/scenarios.json	Scenario library used by the Customer Simulator Agent.
data/knowledge_base/	The FAQ/policy source documents the RAG layer searches.
data/transcripts/	Sample transcripts used by Replay Mode.
data/hall_of_fame.json	Archive of the best- and worst-scoring sessions.

9. tests/

Pytest suite covering the agents, RAG search, and load testing — for verifying code correctness, not something you need to demo live.

Quick one-liner you can say out loud: "The main file is run.py, the UI lives in src/ui/app.py, all the AI logic is split across 24 files in src/agents/, src/core/ handles orchestration and prompts, and every session's score gets saved as a JSON report in data/reports/."